

VTOR 寄存器到底是干什么的？（官方定义 + 真实用途总结）

寄存器全称：Vector Table Offset Register

地址：0xE000ED08

复位值：0x00000000（表示向量表默认放在 0x00000000 地址）

VTOR 的唯一用途：

告诉 Cortex-M 内核“异常向量表（Vector Table）现在被我搬到了哪里”。

异常向量表是一张“异常号 ⇄ 处理函数入口地址”的查找表，CPU 每发生一次异常（复位、HardFault、SVCall、外部中断等），都会到这张表里去取对应的处理函数地址。

如果不改 VTOR，Cortex-M 永远只从下面两个地方找向量表：

- 闪存起始地址 0x00000000（默认，适用于所有芯片上电从 Flash 启动）
- 如果 Boot 引脚配置为从系统存储器（System Memory，如内部 Bootloader）启动，则固定从 0x1FF00000 或 0x1FFF0000 等地址找（此时根本不能改 VTOR）

所以 VTOR 的核心用途只有以下 4 种真实场景

场景 			
序号	实际用途	为什么必须改 VTOR	典型应用场景
1	把程序运行在 SRAM 中	Flash 中的向量表 CPU 根本看不到	烧录新固件（IAP）、高速运行、调试阶段全速跑在 RAM
2	使用 bootloader + app 跳转	Bootloader 完成跳转后，app 的向量表在 Flash 高地址	固件升级、双分区固件、Bootloader + Application
3	在外部并行 NOR/QSPI Flash 上运行代码	外部 Flash 映射到 0x60000000~0x9FFFFFFF 区域	STM32H7、RT1052、RA6M 等带 QSPI/XIP 的芯片
4	多镜像布局或 TrustZone 分区 (M33)	Secure 与 Non-Secure 镜像有各自的向量表	带 TrustZone 的 Cortex-M33 项目

下面按这 4 种场景逐个给出最详细、最真实的 demo（基于 STM32 的真实工程，可直接编译运行）

场景 1：在 SRAM 中运行整个程序（最常见 IAP 第二步）


```

// 假设你已经把 .text、.data 全部拷贝到了 SRAM (0x20000000 开始)
// 现在只需要让 CPU 找到新的向量表即可

extern uint32_t __sram_vectors_start__; // 在 linker script 中定义

void JumpToSRAM(void)
{
    // 1. 关闭全局中断 (防止跳转瞬间有中断打进来用旧向量表)
    __disable_irq();

    // 2. 设置 VTOR 为 SRAM 中向量表的新地址 (必须 512 字节对齐!)
    SCB->VTOR = (uint32_t)&__sram_vectors_start__;

    // 3. 读取新的复位向量 (即新的 main 函数地址)
    uint32_t jump_addr = *((volatile uint32_t *) (SCB->VTOR + 4));

    // 4. 读取新的栈顶地址 (向量表第 0 项)
    uint32_t stack_addr = *((volatile uint32_t *) SCB->VTOR);

    // 5. 跳转
    __set_MSP(stack_addr); // 设置主栈指针
    ((void (*)(void))jump_addr)(); // 跳转到新 ResetHandler
}

// linker script 关键部分 (keil/iar/gcc 通用)
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K
    SRAM (rwx) : ORIGIN = 0x20000000, LENGTH = 512K
}

SECTIONS
{
    .sram_vectors :
    {
        __sram_vectors_start__ = .;
        KEEP(*(.isr_vector)) // 向量表必须放在最前面
        . = ALIGN(512); // 强制 512 字节对齐 (Cortex-M3/M4/M7 要求)
    } > SRAM
}

```

场景 2：Bootloader 跳转到 App（最最最常用的固件升级）

```
C
// Bootloader 位于 0x08000000 ~ 0x0800FFFF (64KB)
// App      位于 0x08010000 开始

#define APP_START_ADDRESS    0x08010000

void JumpToApp(void)
{
    uint32_t app_stack = *((volatile uint32_t *)APP_START_ADDRESS);
    uint32_t app_entry  = *((volatile uint32_t *) (APP_START_ADDRESS + 4));

    __disable_irq();

    // 关键一步：把 VTOR 指向 App 的向量表
    SCB->VTOR = APP_START_ADDRESS & 0xFFFFFE00;    // 自动 512 字节对齐（最低 9 位为 0）

    __set_MSP(app_stack);
    ((void (*)(void))app_entry)();
}
```

Linker script 中 App 的起始地址必须 512 字节对齐（Keil/IAR 直接设置 Flash Origin = 0x08010000，GCC 这样写）：

```
ld
/* gcc linker script */
MEMORY
{
    FLASH (rx) : ORIGIN = 0x08010000, LENGTH = 960K    /* 注意这里是 0x08010000 */
}
```

场景 3：运行在外部 QSPI Flash（STM32H7、RT1052 等）

C



Copy

```
// 外部 QSPI Flash 映射到 0x90000000
#define EXTERNAL_FLASH_VECTORS 0x90000000

void RunFromExternalFlash(void)
{
    SCB->VTOR = EXTERNAL_FLASH_VECTORS;    // 外部 Flash 已经是对齐的
    // 然后直接 return 就行, CPU 已经能正确取向量了
}
```

场景 4: Cortex-M33 Secure/Non-Secure 双向量表 (TrustZone)

C



Copy

```
// Secure 镜像向量表在 0x10000000
// Non-Secure 镜像向量表在 0x10200000

// 在 Secure 代码里切换到 Non-Secure App
SCB_NS->VTOR = 0x10200000;    // 注意是 SCB_NS! (Non-Secure 的 SCB)
SAU->RNR = 2;
SAU->RBAR = 0x10200000;
SAU->RLAR = 0x102FFFFFF | SAU_RLAR_ENABLE_Msk | SAU_RLAR_NSC_Msk;
__TZ_set_SECURECALLGATE_NS(...);    // 具体略
```

总结: 你什么时候必须写 VTOR?

你不写 VTOR

你写了 VTOR



程序永远只能从 0x00000000 启动 程序可以从 Flash 高地址、SRAM、外部 Flash 启动

无法实现 bootloader + app

固件升级变得可能

无法在 RAM 中全速运行

IAP 第二步可以全速调试

无法支持 TrustZone 双镜像

支持 Secure/Non-Secure 分离

一句话: 只要你的向量表不是放在 0x00000000, 你就必须写 VTOR, 否则中断、异常全部飞走, 直接 HardFault。

记住这点，你就彻底掌握了 Cortex-M 的启动精髓。

↳ 向量表详细位字段

↳ NVIC中断配置